



資料結構

Data Structure

Lab12

姓名：李品融

學號：113310217

Lab12-Q1Q2

這題要求我們設計多條件排序演算法，依效期與熱度排列超商商品，並匯入三組測試檔案驗證排序結果。

Code

```
#include <iostream>
#include <vector> // 負責使用動態陣列 (std::vector)，用來儲存未知數量的商品
#include <string> // 負責處理字串型態 (std::string) 與字串切片工具
#include <algorithm> // 負責提供高效的排序演算法 (std::sort)
#include <fstream> // 負責檔案讀寫功能 (std::ifstream)

using namespace std;

// 定義商品的結構
struct Product {
    string name; // 商品名稱
    int expiration; // 剩餘有效期限
    int popularity; // 熱門程度
};

// 比較函式
// 如果兩個商品的效期不同，效期越近的要放在越顯眼的位置，所以使用小於比較
bool compareProducts(const Product& a, const Product& b) {
    if (a.expiration != b.expiration) {
        return a.expiration < b.expiration;
    }

    // 如果兩個商品的效期相同
    // 則看熱門程度，越熱門的要放在越顯眼的位置，所以使用大於比較
    return a.popularity > b.popularity;
}

int main() {
    ifstream infile("input.txt"); // 宣告檔案輸入流物件，並嘗試開啟同資料夾下的
    "input.txt" 檔案

    // 如果路徑錯了或檔案不存在，infile.is_open() 會回傳 false
    if (!infile.is_open()) {
```

```
cerr << "【錯誤】無法開啟 input.txt!" << endl;
cerr << "請確認 input.txt 檔案是否與此程式碼編譯後的 .exe 檔案放在同一個資料夾中。" << endl;
return 1; // 回傳 1 代表程式異常終止
}

int n; // 用來儲存商品總數量
// 從檔案讀取第一個整數（商品數量 N）。如果讀取失敗，就結束程式。
if (!(infile >> n)) return 0;
infile.ignore(); // 忽略數字後的換行符

vector<Product> products; // 宣告一個 Vector 動態陣列，用來放置所有讀入並解析完畢的商品物件

for (int i = 0; i < n; ++i) {
    string line;

    // 使用 getline 讀取檔案的一整行資料（包含字串中間的空格）
    // 如果讀不到東西，就跳出迴圈
    if (!getline(infile, line)) break;

    // 從字串的最尾端開始往前找最後一個空格的位置
    size_t last_space = line.find_last_of(' ');
    if (last_space == string::npos) continue; // 安全防護：找不到空格就跳過這行

    // 擷取最後一個空格後面的子字串，並利用 stoi (String to Integer) 轉成整數
    int popularity = stoi(line.substr(last_space + 1));

    // 切除掉最後的熱度部分，留下剩下的前半段字串
    string remaining = line.substr(0, last_space);

    // 在剩下的字串中，再次從尾端尋找最後一個空格
    size_t second_last_space = remaining.find_last_of(' ');
    if (second_last_space == string::npos) continue; // 安全防護

    // 擷取這個空格後面的子字串，轉成整數
    int expiration = stoi(remaining.substr(second_last_space + 1));

    // 剩下的最前段，不管中間有多少空格，都一定屬於商品的完整名稱
```

```
string name = remaining.substr(0, second_last_space);

products.push_back({name, expiration, popularity}); // 將解析出來的 名稱、效
期、熱度 包裝成 Product 結構，推入 Vector 陣列中
}

// 當所有資料都讀取並放入記憶體後，關閉檔案，釋放系統資源
infile.close();

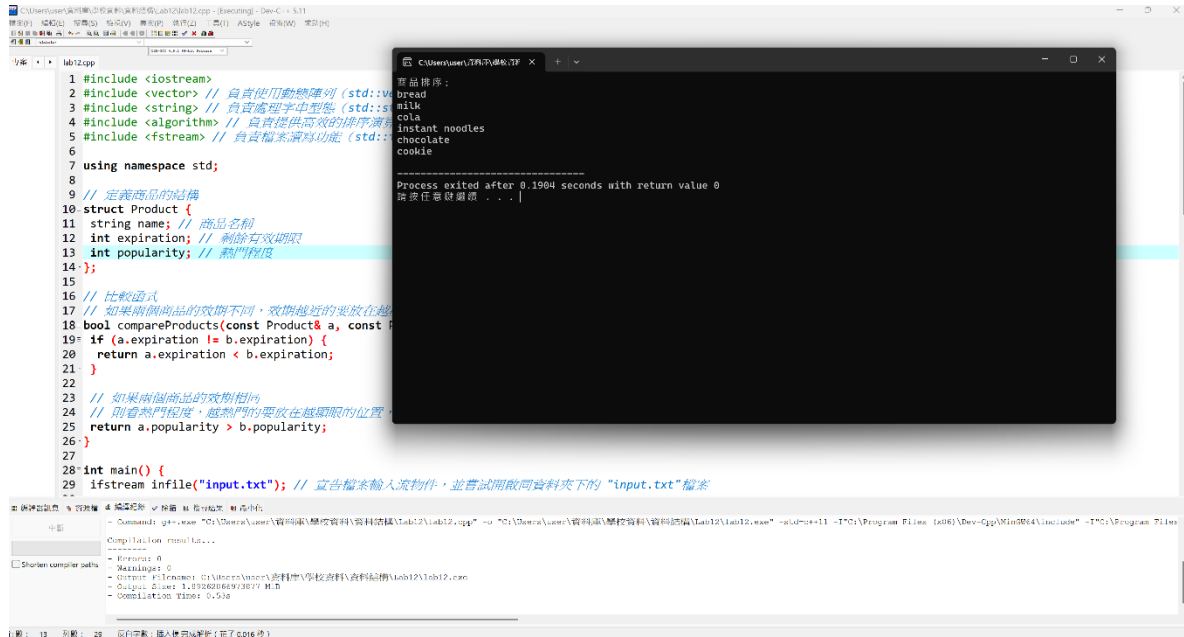
// 呼叫標準庫的高效排序並執行
sort(products.begin(), products.end(), compareProducts);

// 使用 Range-based for Loop (範圍基礎迴圈) 依序取出排序後的每個商品
// const auto& 代表使用唯讀且引用的方式讀取，避免複製結構體，提升效能。
cout << "商品排序:" << endl;
for (const auto& prod : products) {
    cout << prod.name << endl; // 輸出題目要求的商品名稱
}

return 0;
}
```

Result

若輸入檔案為 input1.txt，則：

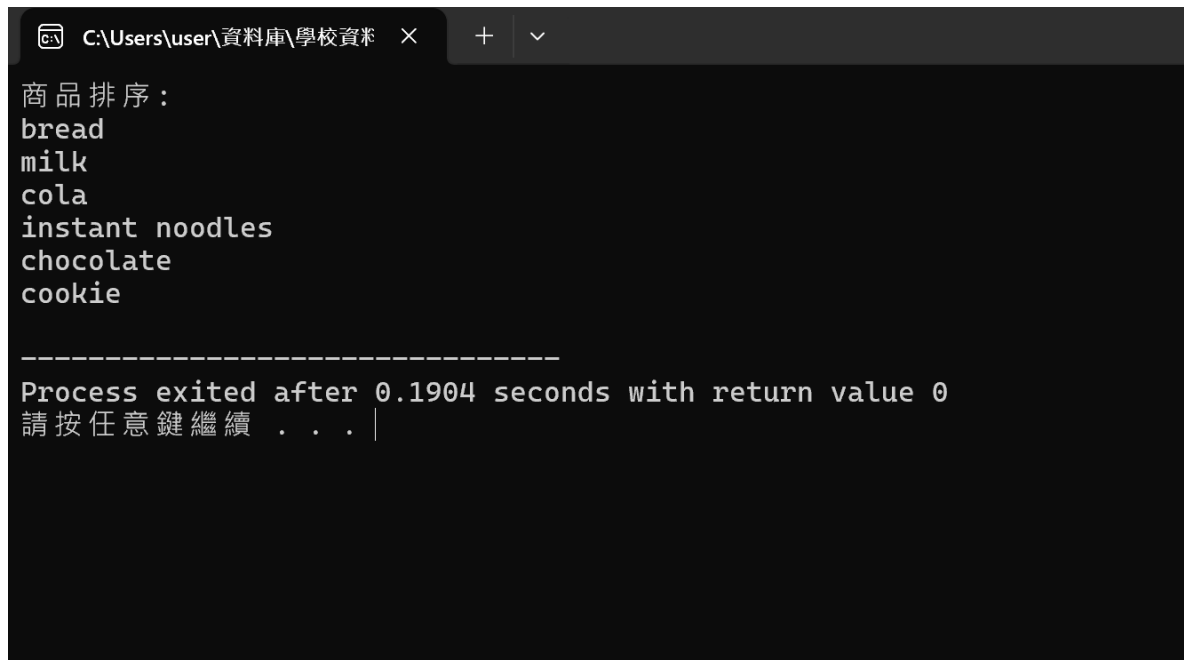


The screenshot shows a C++ IDE with two windows. The left window displays the source code for `lab12.cpp`, which includes headers for `<iostream>`, `<vector>`, `<string>`, `<algorithm>`, and `<fstream>`. It defines a `Product` struct with `name`, `expiration`, and `popularity` attributes. A `compareProducts` function is defined to sort products by expiration date, and if equal, by popularity. The `main` function opens `input.txt` and reads the data. The right window shows the program's output, which lists the sorted products: bread, milk, cola, instant noodles, chocolate, and cookie. Below the list, it states "Process exited after 0.1904 seconds with return value 0" and prompts the user to press any key to continue.

```
1 #include <iostream>
2 #include <vector> // 負責使用動態陣列 (std::vector)
3 #include <string> // 負責處理字串型態 (std::string)
4 #include <algorithm> // 負責提供高效的排序演算法
5 #include <fstream> // 負責檔案讀寫功能 (std::fstream)
6
7 using namespace std;
8
9 // 定義商品的結構
10 struct Product {
11     string name; // 商品名稱
12     int expiration; // 剩餘有效期限
13     int popularity; // 熱門程度
14 };
15
16 // 比較函式
17 // 如果兩個商品的效期不同，效期越短的要放在越前面
18 bool compareProducts(const Product& a, const Product& b) {
19     if (a.expiration != b.expiration) {
20         return a.expiration < b.expiration;
21     }
22     // 如果兩個商品的效期相同
23     // 則看熱門程度，越熱門的要放在越前面的位置
24     return a.popularity > b.popularity;
25 }
26
27
28 int main() {
29     ifstream infile("input.txt"); // 宣告檔案輸入流物件，並嘗試開啟同資料夾下的 "input.txt" 檔案
30     if (!infile.is_open()) {
31         cout << "Error: Cannot open the file 'input.txt'." << endl;
32         return 1;
33     }
34     vector<Product> products;
35     string line;
36     while (getline(infile, line)) {
37         Product p;
38         stringstream ss(line);
39         ss >> p.name;
40         ss >> p.expiration;
41         ss >> p.popularity;
42         products.push_back(p);
43     }
44     sort(products.begin(), products.end(), compareProducts);
45     cout << "商品排序：" << endl;
46     for (const Product& p : products) {
47         cout << p.name << endl;
48     }
49     cout << "Process exited after 0.1904 seconds with return value 0" << endl;
50     cout << "請按任意鍵繼續 . . . |" << endl;
51     cin.get();
52 }
```

```
商品排序：
bread
milk
cola
instant noodles
chocolate
cookie

Process exited after 0.1904 seconds with return value 0
請按任意鍵繼續 . . . |
```

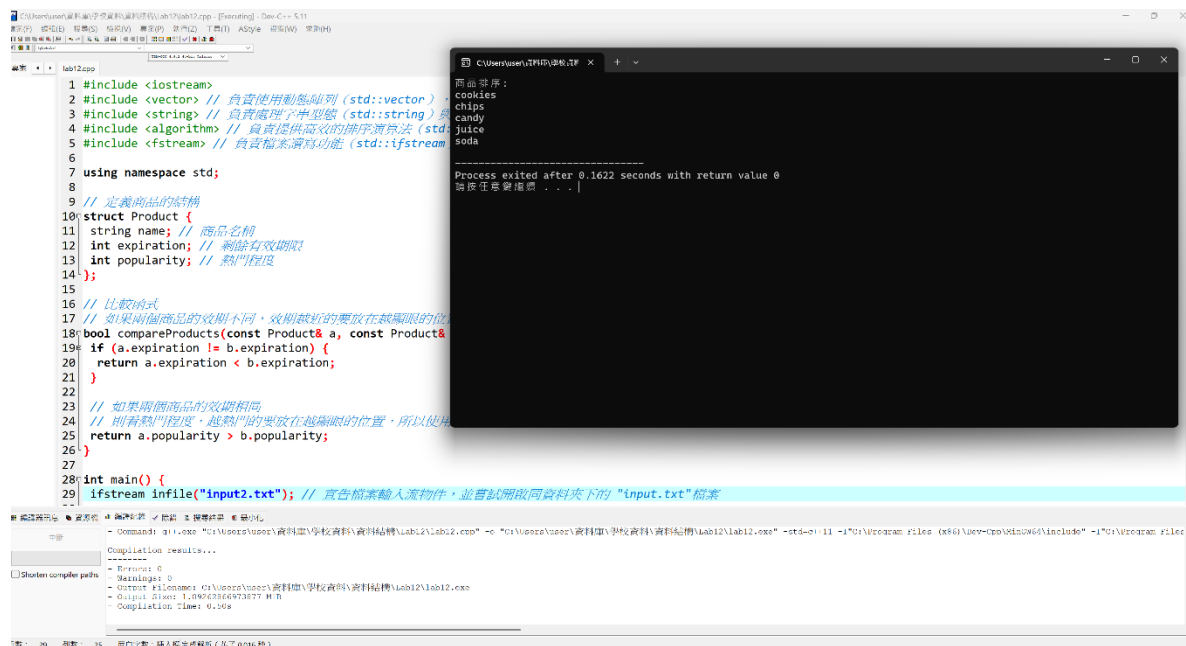


The screenshot shows a terminal window with the same output as the previous one. It lists the sorted products: bread, milk, cola, instant noodles, chocolate, and cookie. Below the list, it states "Process exited after 0.1904 seconds with return value 0" and prompts the user to press any key to continue.

```
商品排序：
bread
milk
cola
instant noodles
chocolate
cookie

Process exited after 0.1904 seconds with return value 0
請按任意鍵繼續 . . . |
```

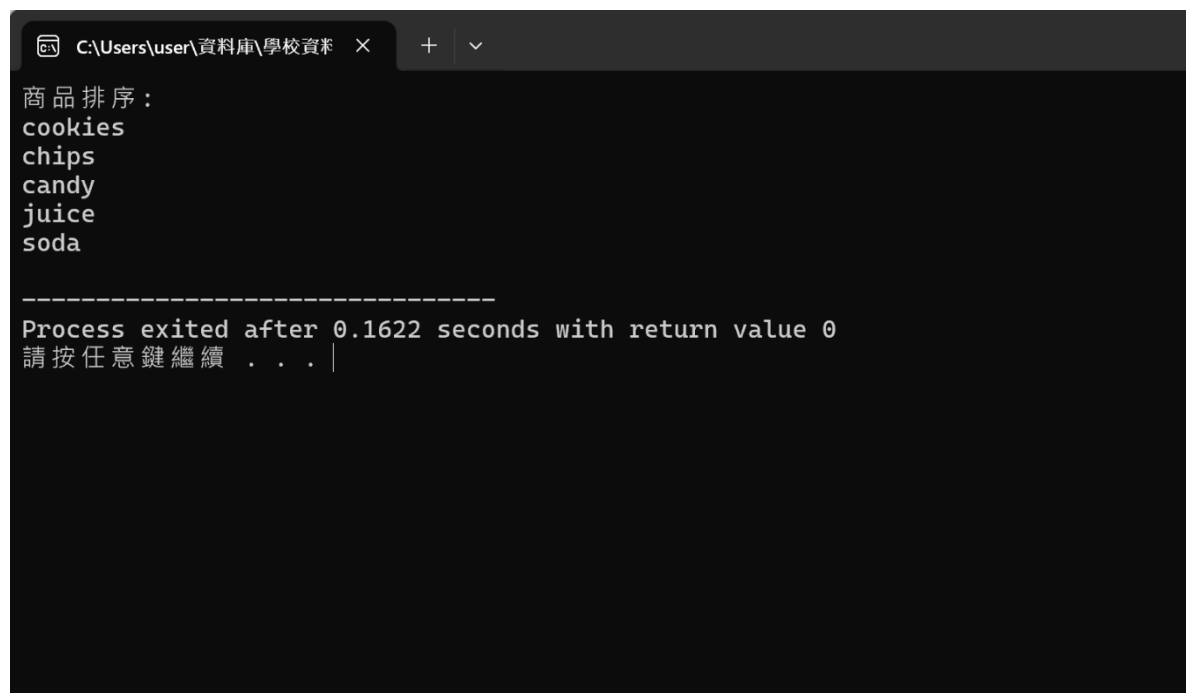
若輸入檔案為 input2.txt，則：



The screenshot shows a C++ IDE with a source file named 'lab12.cpp'. The code defines a 'Product' struct with attributes 'name', 'expiration', and 'popularity'. It includes a comparison function 'compareProducts' that sorts products by expiration date, and if dates are equal, by popularity. The 'main' function reads from 'input2.txt' and prints the sorted list of products.

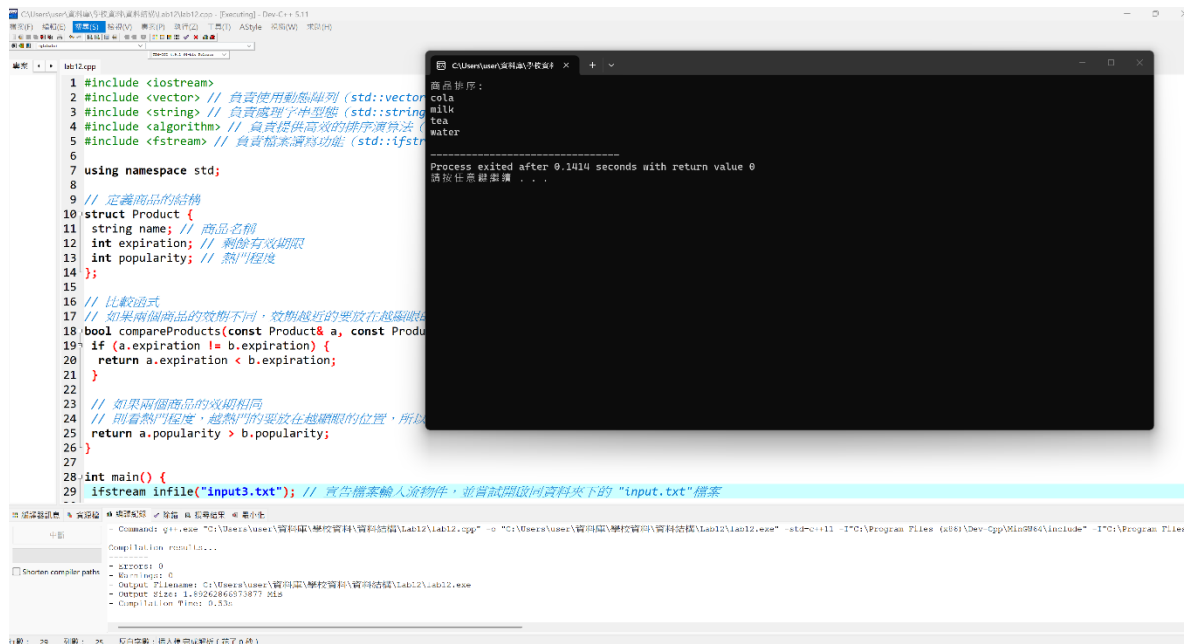
```
1 #include <iostream>
2 #include <vector> // 負責使用動態陣列 (std::vector)
3 #include <string> // 負責處理字串型態 (std::string)
4 #include <algorithm> // 負責提供高效的排序演算法 (std::ifstream)
5 #include <fstream> // 負責檔案讀寫功能 (std::ifstream)
6
7 using namespace std;
8
9 // 定義商品的結構
10 struct Product {
11     string name; // 商品名稱
12     int expiration; // 剩餘有效期限
13     int popularity; // 熱門程度
14 };
15
16 // 比較函式
17 // 如果兩個商品的效期不同，效期越新的要放在越靠前的位置
18 bool compareProducts(const Product& a, const Product& b) {
19     if (a.expiration != b.expiration) {
20         return a.expiration < b.expiration;
21     }
22     // 如果兩個商品的效期相同
23     // 則看熱門程度，越熱門的要放在越靠前的位置，所以使用
24     return a.popularity > b.popularity;
25 }
26
27
28 int main() {
29     ifstream infile("input2.txt"); // 宣告檔案輸入流物件，並嘗試開啟同資料夾下的 "input.txt" 檔案
```

The execution output window shows the sorted list of products: cookies, chips, candy, juice, and soda. It also displays the message: "Process exited after 0.1622 seconds with return value 0" and "請按任意鍵繼續 . . . |".



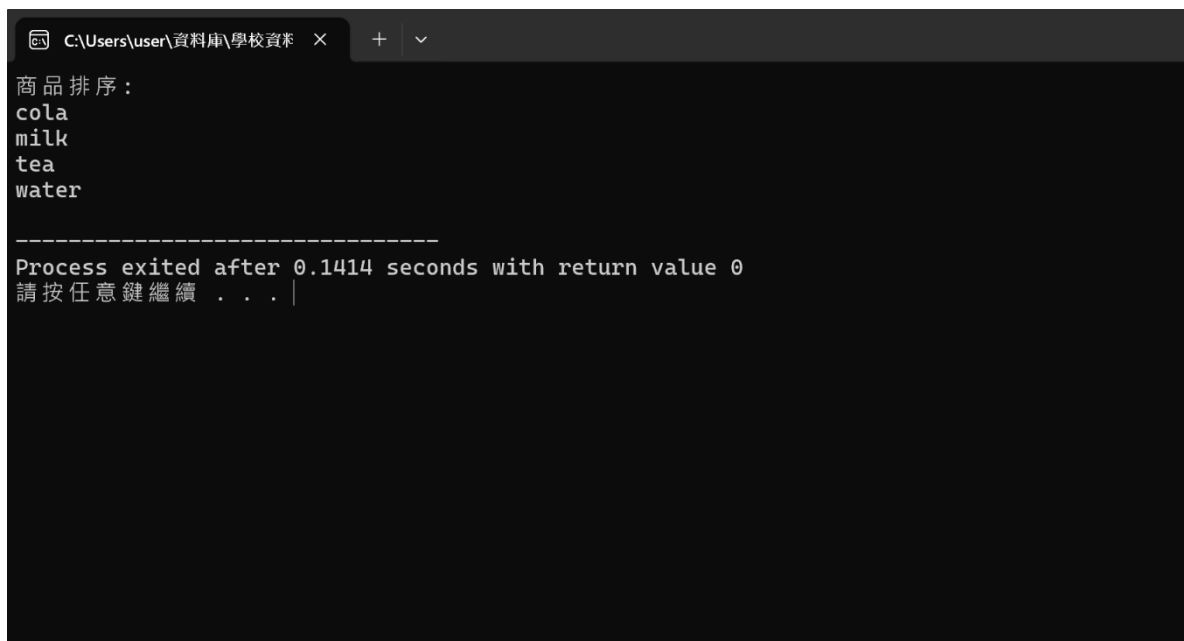
The screenshot shows a terminal window with the same output as the IDE's execution window. It displays the sorted list of products: cookies, chips, candy, juice, and soda. It also displays the message: "Process exited after 0.1622 seconds with return value 0" and "請按任意鍵繼續 . . . |".

若輸入檔案為 input3.txt，則：



```
1 #include <iostream>
2 #include <vector> // 負責使用動態陣列 (std::vector)
3 #include <string> // 負責處理字串型態 (std::string)
4 #include <algorithm> // 負責提供高效的排序演算法 (std::sort)
5 #include <fstream> // 負責檔案讀寫功能 (std::ifstream)
6
7 using namespace std;
8
9 // 定義商品的結構
10 struct Product {
11     string name; // 商品名稱
12     int expiration; // 剩餘有效期限
13     int popularity; // 熱門程度
14 };
15
16 // 比較函式
17 // 如果兩個商品的效期不同，效期越近的要放在越後面
18 bool compareProducts(const Product& a, const Product& b) {
19     if (a.expiration != b.expiration) {
20         return a.expiration < b.expiration;
21     }
22     // 如果兩個商品的效期相同
23     // 則看熱門程度，越熱門的要放在越前面的位置，所以
24     return a.popularity > b.popularity;
25 }
26
27
28 int main() {
29     ifstream infile("input3.txt"); // 宣告檔案輸入物件，並嘗試開啟同資料夾下的 "input.txt" 檔案
30
31     if (!infile.is_open()) {
32         cerr << "Error: Cannot open input file.\n";
33         return 1;
34     }
35
36     vector<Product> products;
37     string line;
38     while (getline(infile, line)) {
39         Product p;
40         if (line.find(',') != string::npos) {
41             p.name = line.substr(0, line.find(','));
42             p.expiration = stoi(line.substr(line.find(',') + 1, line.find(' ')));
43             p.popularity = stoi(line.substr(line.find(' ') + 1));
44             products.push_back(p);
45         }
46     }
47
48     sort(products.begin(), products.end(), compareProducts);
49
50     cout << "商品排序：" << endl;
51     for (const Product& p : products) {
52         cout << p.name << endl;
53     }
54
55     return 0;
56 }
```

Process exited after 0.1414 seconds with return value 0
請按任意鍵繼續 . . .



```
C:\Users\user\資料庫\學校資料 > + v
商品排序：
cola
milk
tea
water

-----
Process exited after 0.1414 seconds with return value 0
請按任意鍵繼續 . . . |
```

Lab12-Q3

這題要求我們分析所選排序演算法的時間與空間複雜度，並從執行效率、記憶體消耗與多條件排序的適用性，闡述選擇該演算法的理由。

Discussion

關於時間複雜度與演算法的部分，我採用 C++ 的內省排序，能確保在任何極端環境下，時間複雜度都穩定維持在高效的 $O(N \log N)$ ，遠優於傳統泡泡排序的 $O(N^2)$ ，適合處理超商龐大的商品庫。而空間複雜度則為精簡的 $O(N)$ ，選擇此演算法除了看中其工業級的執行效率外，更因為它能透過自訂比較函式，在單次排序中同時優雅地處理有效期限與熱度的雙重權重，讓程式碼既乾淨俐落又極具未來的擴充性。